CHAPTER

SIX

INTRODUCTION TO OPENMP

OpenMP has become the *de facto* standard for shared-memory programming.

6.1 Overview

OpenMP has become the environment of choice for many, if not most, practitioners of shared-memory parallel programming. It consists of a set of directives which are added to one's C/C++/FORTRAN code that manipulate threads, without the programmer him/herself having to deal with the threads directly. This way we get "the best of both worlds"—the true parallelism of (nonpreemptive) threads and the pleasure of avoiding the annoyances of threads programming.

Most OpenMP constructs are expressed via **pragmas**, i.e. directives. The syntax is

```
#pragma omp ......
```

The number sign must be the first nonblank character in the line.

6.2 Example: Dijkstra Shortest-Path Algorithm

The following example, implementing Dijkstra's shortest-path graph algorithm, will be used throughout this tutorial, with various OpenMP constructs being illustrated later by modifying this code:

```
1  // Dijkstra.c
2
3  // OpenMP example program:  Dijkstra shortest-path finder in a
4  // bidirectional graph; finds the shortest path from vertex 0 to all
5  // others
6
7  // usage:  dijkstra nv print
```

```
8
9   // where nv is the size of the graph, and print is 1 if graph and min
10  // distances are to be printed out, 0 otherwise
11
12  #include <omp.h>
13
14  // global variables, shared by all threads by default; could placed them
15  // above the "parallel" pragma in dowork()
16
17  int nv,  // number of vertices
18      *notdone, // vertices not checked yet
19      nth,  // number of threads
20      chunk,  // number of vertices handled by each thread
21      md,  // current min over all threads
22      mv,  // vertex which achieves that min
23      largeint = -1;  // max possible unsigned int
24
25  unsigned *ohd,  // 1-hop distances between vertices; "ohd[i][j]" is
26           // ohd[i*nv+j]
27           *mind;  // min distances found so far
28
29  void init(int ac, char **av)
30  {  int i,j,tmp;
31     nv = atoi(av[1]);
32     ohd = malloc(nv*nv*sizeof(int));
33     mind = malloc(nv*sizeof(int));
34     notdone = malloc(nv*sizeof(int));
35     // random graph
36     for (i = 0; i < nv; i++)
37        for (j = i; j < nv; j++)  {
38           if (j == i) ohd[i*nv+i] = 0;
39           else  {
40              ohd[nv*i+j] = rand() % 20;
41              ohd[nv*j+i] = ohd[nv*i+j];
42           }
43        }
44     for (i = 1; i < nv; i++)  {
45        notdone[i] = 1;
46        mind[i] = ohd[i];
47     }
48  }
49
50  // finds closest to 0 among notdone, among s through e
51  void findmymin(int s, int e, unsigned *d, int *v)
52  {  int i;
53     *d = largeint;
54     for (i = s; i <= e; i++)
55        if (notdone[i] && mind[i] < *d)  {
56           *d = mind[i];
57           *v = i;
58        }
59  }
60
61  // for each i in [s,e], ask whether a shorter path to i exists, through
62  // mv
63  void updatemind(int s, int e)
64  {  int i;
65     for (i = s; i <= e; i++)
66        if (mind[mv] + ohd[mv*nv+i] < mind[i])
67           mind[i] = mind[mv] + ohd[mv*nv+i];
68  }
```

```
69
70  void dowork()
71  {
72     #pragma omp parallel
73     {  int startv,endv,  // start, end vertices for my thread
74           step,  // whole procedure goes nv steps
75           mymv,  // vertex which attains the min value in my chunk
76           me = omp_get_thread_num();
77           unsigned mymd;  // min value found by this thread
78        #pragma omp single
79        {  nth = omp_get_num_threads();  // must call from inside parallel block
80           if (nv % nth != 0) {
81              printf("nv must be divisible by nth\n");
82              exit(1);
83           }
84           chunk = nv/nth;
85           printf("there are %d threads\n",nth);
86        }
87        startv = me * chunk;
88        endv = startv + chunk - 1;
89        for (step = 0; step < nv; step++)  {
90           // find closest vertex to 0 among notdone; each thread finds
91           // closest in its group, then we find overall closest
92           #pragma omp single
93           {  md = largeint; mv = 0;  }
94           findmymin(startv,endv,&mymd,&mymv);
95           // update overall min if mine is smaller
96           #pragma omp critical
97           {  if (mymd < md)
98                 {  md = mymd; mv = mymv;  }
99           }
100          #pragma omp barrier
101          // mark new vertex as done
102          #pragma omp single
103          {  notdone[mv] = 0;  }
104          // now update my section of mind
105          updatemind(startv,endv);
106          #pragma omp barrier
107       }
108    }
109 }
110
111 int main(int argc, char **argv)
112 {  int i,j,print;
113    double startime,endtime;
114    init(argc,argv);
115    startime = omp_get_wtime();
116    // parallel
117    dowork();
118    // back to single thread
119    endtime = omp_get_wtime();
120    printf("elapsed time:  %f\n",endtime-startime);
121    print = atoi(argv[2]);
122    if (print)  {
123       printf("graph weights:\n");
124       for (i = 0; i < nv; i++)  {
125          for (j = 0; j < nv; j++)
126             printf("%u  ",ohd[nv*i+j]);
127          printf("\n");
128       }
129       printf("minimum distances:\n");
```

```
130         for (i = 1; i < nv; i++)
131            printf("%u\n",mind[i]);
132      }
133   }
```

The constructs will be presented in the following sections, but first the algorithm will be explained.

6.2.1 The Algorithm

The code implements the Dijkstra algorithm for finding the shortest paths from vertex 0 to the other vertices in an N-vertex undirected graph. Pseudocode for the algorithm is shown below, with the array G assumed to contain the one-hop distances between vertices.

```
1   Done = {0}  # vertices checked so far
2   NewDone = None  # currently checked vertex
3   NonDone = {1,2,...,N-1}  # vertices not checked yet
4   for J = 0 to N-1 Dist[J] = G(0,J)  # initialize shortest-path lengths
5
6   for Step = 1 to N-1
7      find J such that Dist[J] is min among all J in NonDone
8      transfer J from NonDone to Done
9      NewDone = J
10     for K = 1 to N-1
11        if K is in NonDone
12           # check if there is a shorter path from 0 to K through NewDone
13           # than our best so far
14           Dist[K] = min(Dist[K],Dist[NewDone]+G[NewDone,K])
```

At each iteration, the algorithm finds the closest vertex J to 0 among all those not yet processed, and then updates the list of minimum distances to each vertex from 0 by considering paths that go through J. Two obvious potential candidate part of the algorithm for parallelization are the "find J" and "for K" lines, and the above OpenMP code takes this approach.

6.2.2 The OpenMP `parallel` Pragma

As can be seen in the comments in the lines

```
// parallel
dowork();
// back to single thread
```

the function **main()** is run by a **master thread**, which will then branch off into many threads running **dowork()** in parallel. The latter feat is accomplished by the directive in the lines

```
void dowork()
{
   #pragma omp parallel
   {  int startv,endv,  // start, end vertices for this thread
          step,  // whole procedure goes nv steps
          mymv,  // vertex which attains that value
          me = omp_get_thread_num();
```

That directive sets up a team of threads (which includes the master), all of which execute the block following the directive in parallel.[1] Note that, unlike the **for** directive which will be discussed below, the **parallel** directive leaves it up to the programmer as to how to partition the work. In our case here, we do that by setting the range of vertices which this thread will process:

```
      startv = me * chunk;
      endv = startv + chunk - 1;
```

Again, keep in mind that *all* of the threads execute this code, but we've set things up with the variable **me** so that different threads will work on different vertices. This is due to the OpenMP call

```
          me = omp_get_thread_num();
```

which sets **me** to the thread number for this thread.

6.2.3 Scope Issues

Note carefully that in

```
   #pragma omp parallel
   {  int startv,endv,  // start, end vertices for this thread
          step,  // whole procedure goes nv steps
          mymv,  // vertex which attains that value
          me = omp_get_thread_num();
```

the pragma comes *before* the declaration of the local variables. That means that all of them are "local" to each thread, i.e. not shared by them. But if a work sharing directive comes within a function but *after* declaration of local variables, those variables are actually "global" to the code in the directive, i.e. they *are* shared in common among the threads.

This is the default, but you can change these properties, e.g. using the **private** keyword and its cousins. For instance,

[1] There is an issue here of thread startup time. The OMPi compiler sets up threads at the outset, so that that startup time is incurred only once. When a **parallel** construct is encountered, they are awakened. At the end of the construct, they are suspended again, until the next **parallel** construct is reached.

```
#pragma omp parallel private(x,y)
```

would make **x** and **y** nonshared even if they were declared above the directive line. You may wish to modify that a bit, so that **x** and **y** have initial values that were shared before the directive; use **firstprivate** for this.

It is crucial to keep in mind that variables which are global to the program (in the C/C++ sense) are automatically global to all threads. This is the primary means by which the threads communicate with each other.

6.2.4 The OpenMP `single` Pragma

In some cases we want just one thread to execute some code, even though that code is part of a **parallel** or other **work sharing** block.[2] We use the **single** directive to do this, e.g.:

```
#pragma omp single
{  nth = omp_get_num_threads();
   if (nv % nth != 0) {
      printf("nv must be divisible by nth\n");
      exit(1);
   }
   chunk = nv/nth;
   printf("there are %d threads\n",nth);  }
```

Since the variables **nth** and **chunk** are global and thus shared, we need not have all threads set them, hence our use of **single**.

6.2.5 The OpenMP `barrier` Pragma

As see in the example above, the **barrier** implements a standard barrier, applying to all threads.

6.2.6 Implicit Barriers

Note that there is an implicit barrier at the end of each **single** block, which is also the case for **parallel**, **for**, and **sections** blocks. This can be overridden via the **nowait** clause, e.g.

```
#pragma omp for nowait
```

Needless to say, the latter should be used with care, and in most cases will not be usable. On the other hand, putting in a barrier where it is not needed would severely reduce performance.

[2] This is an OpenMP term. The **for** directive is another example of it. More on this below.

6.2.7 The OpenMP `critical` Pragma

The last construct used in this example is **critical**, for critical sections.

```
#pragma omp critical
{  if (mymd < md)
     {  md = mymd; mv = mymv;  }
}
```

It means what it says, allowing entry of only one thread at a time while others wait. Here we are updating global variables **md** and **mv**, which has to be done atomically, and **critical** takes care of that for us. This is much more convenient than setting up lock variables, etc., which we would do if we were programming threads code directly.

6.3 The OpenMP `for` Pragma

This one breaks up a C/C++ **for** loop, assigning various iterations to various threads. (The threads, of course, must have already been set up via the **omp parallel** pragma.) This way the iterations are done in parallel. Of course, that means that they need to be independent iterations, i.e. one iteration cannot depend on the result of another.

6.3.1 Example: Dijkstra with Parallel for Loops

Here's how we could use this construct in the Dijkstra program :

```
1  // Dijkstra.c
2
3  // OpenMP example program (OMPi version):  Dijkstra shortest-path finder
4  // in a bidirectional graph; finds the shortest path from vertex 0 to
5  // all others
6
7  // usage:  dijkstra nv print
8
9  // where nv is the size of the graph, and print is 1 if graph and min
10 // distances are to be printed out, 0 otherwise
11
12 #include <omp.h>
13
14 // global variables, shared by all threads by default
15
16 int nv,  // number of vertices
17     *notdone, // vertices not checked yet
18     nth,  // number of threads
19     chunk,  // number of vertices handled by each thread
20     md,  // current min over all threads
21     mv,  // vertex which achieves that min
22     largeint = -1;  // max possible unsigned int
23
24 unsigned *ohd,  // 1-hop distances between vertices; "ohd[i][j]" is
```

```
25                 // ohd[i*nv+j]
26          *mind;  // min distances found so far
27
28  void init(int ac, char **av)
29  {  int i,j,tmp;
30     nv = atoi(av[1]);
31     ohd = malloc(nv*nv*sizeof(int));
32     mind = malloc(nv*sizeof(int));
33     notdone = malloc(nv*sizeof(int));
34     // random graph
35     for (i = 0; i < nv; i++)
36        for (j = i; j < nv; j++)  {
37           if (j == i) ohd[i*nv+i] = 0;
38           else  {
39              ohd[nv*i+j] = rand() % 20;
40              ohd[nv*j+i] = ohd[nv*i+j];
41           }
42        }
43     for (i = 1; i < nv; i++)  {
44        notdone[i] = 1;
45        mind[i] = ohd[i];
46     }
47  }
48
49  void dowork()
50  {
51     #pragma omp parallel
52     {  int step,  // whole procedure goes nv steps
53           mymv,  // vertex which attains that value
54           me = omp_get_thread_num(),
55           i;
56        unsigned mymd;  // min value found by this thread
57        #pragma omp single
58        {  nth = omp_get_num_threads();
59           printf("there are %d threads\n",nth);  }
60        for (step = 0; step < nv; step++)  {
61           // find closest vertex to 0 among notdone; each thread finds
62           // closest in its group, then we find overall closest
63           #pragma omp single
64           {  md = largeint; mv = 0;  }
65           mymd = largeint;
66           #pragma omp for nowait
67           for (i = 1; i < nv; i++)  {
68              if (notdone[i] && mind[i] < mymd)  {
69                 mymd = ohd[i];
70                 mymv = i;
71              }
72           }
73           // update overall min if mine is smaller
74           #pragma omp critical
75           {  if (mymd < md)
76                {  md = mymd; mv = mymv;  }
77           }
78           // mark new vertex as done
79           #pragma omp single
80           {  notdone[mv] = 0;  }
81           // now update ohd
82           #pragma omp for
83           for (i = 1; i < nv; i++)
84              if (mind[mv] + ohd[mv*nv+i] < mind[i])
85                 mind[i] = mind[mv] + ohd[mv*nv+i];
```

```
86          }
87       }
88    }
89
90    int main(int argc, char **argv)
91    {  int i,j,print;
92       init(argc,argv);
93       // parallel
94       dowork();
95       // back to single thread
96       print = atoi(argv[2]);
97       if (print)  {
98          printf("graph weights:\n");
99          for (i = 0; i < nv; i++)  {
100            for (j = 0; j < nv; j++)
101               printf("%u  ",ohd[nv*i+j]);
102            printf("\n");
103         }
104         printf("minimum distances:\n");
105         for (i = 1; i < nv; i++)
106            printf("%u\n",mind[i]);
107      }
108   }
109
```

The work which used to be done in the function **findmymin()** is now done here:

```
#pragma omp for
for (i = 1; i < nv; i++)  {
   if (notdone[i] && mind[i] < mymd)  {
      mymd = ohd[i];
      mymv = i;
   }
}
```

Each thread executes one or more of the iterations, i.e. takes responsibility for one or more values of i. This occurs in parallel, so as mentioned earlier, the programmer must make sure that the iterations are independent; there is no predicting which threads will do which values of **i**, in which order. By the way, for obvious reasons OpenMP treats the loop index, **i** here, as private even if by context it would be shared.

6.3.2 Nested Loops

If we use the **for** pragma to nested loops, by default the pragma applies only to the outer loop. We can of course insert another **for** pragma inside, to parallelize the inner loop.

Or, starting with OpenMP version 3.0, one can use the **collapse** clause, e.g.

```
#pragma omp parallel for collapse(2)
```

to specify two levels of nesting in the assignment of threads to tasks.

6.3.3 Controlling the Partitioning of Work to Threads: the schedule Clause

In this default version of the **for** construct, iterations are executed by threads *in unpredictable order*; the OpenMP standard does not specify which threads will execute which iterations in which order. But this can be controlled by the programmer, using the **schedule** clause. OpenMP provides three choices for this:

- **static:** The iterations are grouped into chunks, and assigned to threads in round-robin fashion. Default chunk size is approximately the number of iterations divided by the number of threads.
- **dynamic:** Again the iterations are grouped into chunks, but here the assignment of chunks to threads is done dynamically. When a thread finishes working on a chunk, it asks the OpenMP runtime system to assign it the next chunk in the queue. Default chunk size is 1.
- **guided:** Similar to dynamic, but with the chunk size decreasing as execution proceeds.

For instance, our original version of our program in Section 6.2 broke the work into chunks, with chunk size being the number vertices divided by the number of threads.

For the Dijkstra algorithm, for instance, we could get the same operation with less code by asking OpenMP to do the chunking for us.

```
...
        #pragma omp for schedule(static)
        for (i = 1; i < nv; i++)  {
           if (notdone[i] && mind[i] < mymd)  {
              mymd = ohd[i];
              mymv = i;
           }
        }
...
        #pragma omp for schedule(static)
        for (i = 1; i < nv; i++)
           if (mind[mv] + ohd[mv*nv+i] < mind[i])
              mind[i] = mind[mv] + ohd[mv*nv+i];
...
```

Note again that this would have the same effect as our original code, which each thread handling one chunk of contiguous iterations within a loop. So it's just a programming convenience for us in this case. (If the number of threads doesn't evenly divide the number of iterations, OpenMP will fix that up for us too.)

The more general form is

```
#pragma omp for schedule(static,chunk)
```

Here **static** is still a keyword but **chunk**, specifying the chunk size, is an actual argument. However, setting the chunk size in the **schedule()** clause is a *compile-time* operation. If you wish to have the chunk size set at run time, call **omp_set_schedule()** in conjunction with the **runtime** clause. Example:

```
1  int main(int argc, char **argv)
2  {
3     ...
4     n = atoi(argv[1]);
5     int chunk = atoi(argv[2]);
6     omp_set_schedule(omp_sched_static, chunk);
7     #pragma omp parallel
8     {
9        ...
10       #pragma omp for schedule(runtime)
11       for (i = 1; i < n; i++)  {
12          ...
13       }
14       ...
15    }
16 }
```

Or set the **OMP_SCHEDULE** environment variable. The syntax is the same for **dynamic** and **guided**, e.g.

```
1  setenv OMP_SCHEDULE "static,20"
```

As discussed in Section 21.4, on the one hand, large chunks are good, due to there being less overhead—every time a thread finishes a chunk, it must go through the critical section, which serializes our parallel program and thus slows things down. On the other hand, if chunk sizes are large, then toward the end of the work, some threads may be working on their last chunks while others have finished and are now idle, thus foregoing potential speed enhancement. So it would be nice to have large chunks at the beginning of the run, to reduce the overhead, but smaller chunks at the end. This can be done using the **guided** clause.

For the Dijkstra algorithm, for instance, we could have this:

```
...
        #pragma omp for schedule(guided)
        for (i = 1; i < nv; i++)  {
           if (notdone[i] && mind[i] < mymd)  {
              mymd = ohd[i];
```

```
            mymv = i;
         }
      }
...
      #pragma omp for schedule(guided)
      for (i = 1; i < nv; i++)
         if (mind[mv] + ohd[mv*nv+i] < mind[i])
            mind[i] = mind[mv] + ohd[mv*nv+i];
...
```

There are other variations of this available in OpenMP. However, in Section 21.4, I showed that these would seldom be necessary or desirable; having each thread handle a single chunk would be best.

See Section 21.4 for a timing example.

6.3.4 Example: In-Place Matrix Transpose

This method works in-place, a virtue if we are short on memory. Its cache performance is probably poor, though. It may be better to look at horizontal slabs above the diagonal, say, and trade them with vertical ones below the diagonal.

```
1  #include <omp.h>
2
3  // translate from 2-D to 1-D indices
4  int onedim(int n,int i,int j) {  return n * i + j;  }
5
6  void transp(int *m, int n)
7  {
8     #pragma omp parallel
9     {  int i,j,tmp;
10       // walk through all the above-diagonal elements, swapping them
11       // with their below-diagonal counterparts
12       #pragma omp for
13       for (i = 0; i < n; i++) {
14          for (j = i+1; j < n; j++) {
15             tmp = m[onedim(n,i,j)];
16             m[onedim(n,i,j)] = m[onedim(n,j,i)];
17             m[onedim(n,j,i)] = tmp;
18          }
19       }
20    }
21 }
```

6.3.5 The OpenMP `reduction` Clause

The name of this OpenMP clause alludes to the term **reduction** in functional programming. Many parallel programming languages include such operations, to enable the programmer to more conveniently (and often more efficiently) have threads/processors cooperate in computing sums, products, etc. OpenMP does this via the **reduction** clause.

For example, consider

```
1  int z;
2  ...
3  #pragma omp for reduction(+:z)
4  for (i = 0; i < n; i++)  z += x[i];
```

The pragma says that the threads will share the work as in our previous discussion of the **for** pragma. In addition, though, there will be independent copies of **z** maintained for each thread, each initialized to 0 before the loop begins. When the loop is entirely done, the values of **z** from the various threads will be summed, of course in an atomic manner.

Note that the + operator not only indicates that the values of **z** are to be summed, but also that their initial values are to be 0. If the operator were *, say, then the product of the values would be computed, and their initial values would be 1.

One can specify several reduction variables to the right of the colon, separated by commas.

Our use of the **reduction** clause here makes our programming much easier. Indeed, if we had old serial code that we wanted to parallelize, we would have to make no change to it! OpenMP is taking care of both the work splitting across values of **i**, and the atomic operations. Moreover—note this carefully—it is efficient, because by maintaining separate copies of **z** until the loop is done, we are reducing the number of serializing atomic actions, and are avoiding time-costly cache coherency transactions and the like.

Without this construct, we would have to do

```
int z,myz=0;
...
#pragma omp for private(myz)
for (i = 0; i < n; i++)  myz += x[i];
#pragma omp critical
{ z += myz; }
```

Here are the eligible operators and the corresponding initial values:

In C/C++, you can use **reduction** with +, -, *, &, |, && and || (and the exclusive-or operator).

operator	initial value
+	0
-	0
*	1
&	bit string of 1s
|	bit string of 0s
^	0
&&	1
||	0

The lack of other operations typically found in other parallel programming languages, such as min and max, is due to the lack of these operators in C/C++. The FORTRAN version of OpenMP does have min and max.[3]

Note that the reduction variables must be shared by the threads, and apparently the only acceptable way to do so in this case is to declare them as global variables.

A reduction variable must be scalar, in C/C++. It can be an array in FORTRAN.

6.4 Example: Mandelbrot Set

Here's the code for the timings in Section 21.4.5:

```
1  // compile with -D, e.g.
2  //
3  //    gcc -fopenmp -o manddyn Gove.c -DDYNAMIC
4  //
5  // to get the version that uses dynamic scheduling
6
7  #include <omp.h>
8  #include <complex.h>
9
10 #include <time.h>
11 float timediff(struct timespec t1, struct timespec t2)
12 {  if (t1.tv_nsec > t2.tv_nsec) {
13           t2.tv_sec -= 1;
14           t2.tv_nsec += 1000000000;
15    }
16    return t2.tv_sec-t1.tv_sec + 0.000000001 * (t2.tv_nsec-t1.tv_nsec);
17 }
```

[3] Note, though, that plain min and max would not help in our Dijkstra example above, as we not only need to find the minimum value, but also need the vertex which attains that value.

```
18
19 #ifdef RC
20 // finds chunk among 0,...,n-1 to assign to thread number me among nth
21 // threads
22 void findmyrange(int n,int nth,int me,int *myrange)
23 {  int chunksize = n / nth;
24    myrange[0] = me * chunksize;
25    if (me < nth-1) myrange[1] = (me+1) * chunksize - 1;
26    else myrange[1] = n - 1;
27 }
28
29 #include <stdlib.h>
30 #include <stdio.h>
31 // from http://www.cis.temple.edu/~ingargio/cis71/code/randompermute.c
32 // It returns a random permutation of 0..n-1
33 int * rpermute(int n) {
34   int *a = (int *)(int *)  malloc(n*sizeof(int));
35   // int *a = malloc(n*sizeof(int));
36   int k;
37   for (k = 0; k < n; k++)
38     a[k] = k;
39   for (k = n-1; k > 0; k--) {
40     int j = rand() % (k+1);
41     int temp = a[j];
42     a[j] = a[k];
43     a[k] = temp;
44   }
45   return a;
46 }
47 #endif
48
49 #define MAXITERS 1000
50
51 // globals
52 int count = 0;
53 int nptsside;
54 float side2;
55 float side4;
56
57 int inset(double complex c) {
58    int iters;
```

```
59    float rl,im;
60    double complex z = c;
61    for (iters = 0; iters < MAXITERS; iters++) {
62       z = z*z + c;
63       rl = creal(z);
64       im = cimag(z);
65       if (rl*rl + im*im > 4) return 0;
66    }
67    return 1;
68 }
69
70 int *scram;
71
72 void dowork()
73 {
74    #ifdef RC
75    #pragma omp parallel reduction(+:count)
76    #else
77    #pragma omp parallel
78    #endif
79    {
80       int x,y; float xv,yv;
81       double complex z;
82       #ifdef STATIC
83       #pragma omp for reduction(+:count) schedule(static)
84       #elif defined DYNAMIC
85       #pragma omp for reduction(+:count) schedule(dynamic)
86       #elif defined GUIDED
87       #pragma omp for reduction(+:count) schedule(guided)
88       #endif
89       #ifdef RC
90       int myrange[2];
91       int me = omp_get_thread_num();
92       int nth = omp_get_num_threads();
93       int i;
94       findmyrange(nptsside,nth,me,myrange);
95       for (i = myrange[0]; i <= myrange[1]; i++) {
96          x = scram[i];
97       #else
98       for (x=0; x<nptsside; x++) {
99       #endif
```

```
100           for ( y=0; y<nptsside; y++)  {
101              xv = (x - side2) / side4;
102              yv = (y - side2) / side4;
103              z = xv + yv*I;
104              if (inset(z)) {
105                 count++;
106              }
107           }
108        }
109     }
110  }
111
112  int main(int argc, char **argv)
113  {
114     nptsside = atoi(argv[1]);
115     side2 = nptsside / 2.0;
116     side4 = nptsside / 4.0;
117
118     struct timespec bgn,nd;
119     clock_gettime(CLOCK_REALTIME, &bgn);
120
121     #ifdef RC
122     scram = rpermute(nptsside);
123     #endif
124
125     dowork();
126
127     // implied barrier
128     printf("%d\n",count);
129     clock_gettime(CLOCK_REALTIME, &nd);
130     printf("%f\n",timediff(bgn,nd));
131  }
```

The code is similar to that of a number of books and Web sites, such as the Gove book cited in Section 21.2. Here RC is the random chunk method discussed in Section 21.4.

6.5 The Task Directive

The basic idea is to set up a task queue: When a thread encounters a **task** directive, it arranges for some thread to execute the associated block—at some time. The first thread can continue. Note that the task might not execute right away; it may have to wait for some thread to become free after finishing another task. Also, there may be more tasks than threads, also causing some threads to wait.

Note that we could arrange for all this ourselves, without **task**. We'd set up our own work queue, as a shared variable, and write our code so that whenever a thread finished a unit of work, it would delete the head of the queue. Whenever a thread generated a unit of work, it would add it to the queue. Of course, the deletion and addition would have to be done atomically. All this would amount to a lot of coding on our part, so **task** really simplifies the programming.

6.5.1 Example: Quicksort

```
1  // OpenMP example program:  quicksort; not necessarily efficient
2
3  void swap(int *yi, int *yj)
4  {  int tmp = *yi;
5     *yi = *yj;
6     *yj = tmp;
7  }
8
9  int separate(int *x, int low, int high)
10 {  int i,pivot,last;
11    pivot = x[low];  // would be better to take, e.g., median of 1st 3 elts
12    swap(x+low,x+high);
13    last = low;
14    for (i = low; i < high; i++) {
15       if (x[i] <= pivot) {
16          swap(x+last,x+i);
17          last += 1;
18       }
19    }
20    swap(x+last,x+high);
21    return last;
22 }
23
24 // quicksort of the array z, elements zstart through zend; set the
25 // latter to 0 and m-1 in first call, where m is the length of z;
26 // firstcall is 1 or 0, according to whether this is the first of the
27 // recursive calls
28 void qs(int *z, int zstart, int zend, int firstcall)
29 {
30    #pragma omp parallel
31    {  int part;
32       if (firstcall == 1) {
33          #pragma omp single nowait
34          qs(z,0,zend,0);
35       } else {
```

```
36          if (zstart < zend) {
37             part = separate(z,zstart,zend);
38             #pragma omp task
39             qs(z,zstart,part-1,0);
40             #pragma omp task
41             qs(z,part+1,zend,0);
42          }
43
44       }
45    }
46 }
47
48 // test code
49 main(int argc, char**argv)
50 {  int i,n,*w;
51    n = atoi(argv[1]);
52    w = malloc(n*sizeof(int));
53    for (i = 0; i < n; i++) w[i] = rand();
54    qs(w,0,n-1,1);
55    if (n < 25)
56       for (i = 0; i < n; i++) printf("%d\n",w[i]);
57 }
```

The code

```
if (firstcall == 1) {
   #pragma omp single nowait
   qs(z,0,zend,0);
```

gets things going. We want only one thread to execute the root of the recursion tree, hence the need for the **single** clause. After that, the code

```
part = separate(z,zstart,zend);
#pragma omp task
qs(z,zstart,part-1,0);
```

sets up a call to a subtree, with the **task** directive stating, "OMP system, please make sure that this subtree is handled by some thread eventually."

There are various refinements, such as the barrier-like **taskwait** clause.

6.6 Other OpenMP Synchronization Issues

Earlier we saw the **critical** and **barrier** constructs. There is more to discuss, which we do here.

6.6.1 The OpenMP `atomic` Clause

The **critical** construct not only serializes your program, but also it adds a lot of overhead. If your critical section involves just a one-statement update to a shared variable, e.g.

```
x += y;
```

etc., then the OpenMP compiler can take advantage of an atomic hardware instruction, e.g. the LOCK prefix on Intel, to set up an extremely efficient critical section, e.g.

```
#pragma omp atomic
x += y;
```

Since it is a single statement rather than a block, there are no braces.

The eligible operators are:

```
++, --, +=, *=, <<=, &=, |=
```

Here is an example. The code in Section 6.12 includes an instance of this pragma:

```
for (i = me; i < n; i += nth) {
   mysum += procpairs(i);
}
#pragma omp atomic
tot += mysum;
#pragma omp barrier
```

The compiler produces the following code from that snippet:[4]

```
movl    n(%rip), %eax
cmpl    %eax, -8(%rbp)
jl      .L22
movl    -12(%rbp), %eax
lock addl       %eax, tot(%rip)
call    GOMP_barrier
jmp     .L24
```

Recall that in (Unix-family) Intel assembly language, a percent sign indicates a CPU register, e.g. the EAX register.

The first couple of lines are part of the compiled version of the **for** loop, comparing **i** to **n** and jumping to the top of the look if **i** is still less than **n**.

The **movl** then fills the EAX register with **mysum**, which the compiler has apparently assigned to the location 12 bytes above the place in memory pointed to by the RBP register. The instructions

```
lock addl       %eax, tot(%rip)
```

[4] This can be obtained by compiling with **gcc** with the **-S** option, producing a **.s** file.

atomically adds **mysum** to **tot**.

6.6.2 Memory Consistency and the `flush` Pragma

Consider a shared-memory multiprocessor system with coherent caches, and a shared, i.e. global, variable **x**. If one thread writes to **x**, you might think that the cache coherency system will ensure that the new value is visible to other threads. But as discussed in Section 4.6, it is is not quite so simple as this.

For example, the compiler may store **x** in a register, and update **x** itself at certain points. In between such updates, since the memory location for **x** is not written to, the cache will be unaware of the new value, which thus will not be visible to other threads. If the processors have write buffers etc., the same problem occurs.

In other words, we must account for the fact that our program could be run on different kinds of hardware with different memory consistency models. Thus OpenMP must have its own memory consistency model, which is then translated by the compiler to mesh with the hardware.

OpenMP takes a **relaxed consistency** approach, meaning that it forces updates to memory ("flushes") at all synchronization points, i.e. at:

- **barrier**
- entry/exit to/from **critical**
- entry/exit to/from **ordered**
- entry/exit to/from **parallel**
- exit from **parallel for**
- exit from **parallel sections**
- exit from **single**

In between synchronization points, one can force an update to **x** via the **flush** pragma:

```
#pragma omp flush (x)
```

The flush operation is obviously architecture-dependent. OpenMP compilers will typically have the proper machine instructions available for some common architectures. For the rest, it can force a flush at the hardware level by doing lock/unlock operations, though this may be costly in terms of time.

6.7 Combining Work-Sharing Constructs

In our examples of the **for** pragma above, that pragma would come within a block headed by a **parallel** pragma. The latter specifies that a team of theads is to be created, with each one executing the given block, while the former specifies that the various iterations of the loop are to be distributed among the threads. As a shortcut, we can combine the two pragmas:

```
#pragma omp parallel for
```

This also works with the **sections** pragma.

6.8 The Rest of OpenMP

There is much, much more to OpenMP than what we have seen here. To see the details, there are many Web pages you can check, and there is also the excellent book, *Using OpenMP: Portable Shared Memory Parallel Programming*, by Barbara Chapman, Gabriele Jost and Ruud Van Der Pas, MIT Press, 2008. The book by Gove cited in Section 21.2 also includes coverage of OpenMP.

6.9 Compiling, Running and Debugging OpenMP Code

6.9.1 Compiling

There are a number of open source compilers available for OpenMP, including:

- Omni: This is available at (`http://www.hpcs.cs.tsukuba.ac.jp/omni-compiler/`). To compile an OpenMP program in **x.c** and create an executable file **x**, run

  ```
  omcc -g -o x x.c
  ```

  Note: Apparently declarations of local variables cannot be made in the midst of code; they must precede all code within a block.

- Ompi: You can download this at `http://www.cs.uoi.gr/~ompi/index.html`. Compile **x.c** by

  ```
  ompicc -g -o x x.c
  ```

- GCC, version 4.2 or later:[5] Compile **x.c** via

```
gcc -fopenmp -g -o x x.c
```

  You can also use **-lgomp** instead of **-fopenmp**.

6.9.2 Running

Just run the executable as usual.

The number of threads will be the number of processors, by default. To change that value, set the OMP_NUM_THREADS environment variable. For example, to get four threads in the C shell, type

```
setenv OMP_NUM_THREADS 4
```

6.9.3 Debugging

Since OpenMP is essentially just an interface to threads, your debugging tool's threads facilities should serve you well. See Section **??** for the GDB case.

A possible problem, though, is that OpenMP's use of pragmas makes it difficult for the compilers to maintain your original source code line numbers, and your function and variable names. But with a little care, a symbolic debugger such as GDB can still be used. Here are some tips for the compilers mentioned above, using GDB as our example debugging tool:

- GCC: GCC maintains line numbers and names well. In earlier versions, it had a problem in that it did not not retain names of local variables within blocks controlled by **omp parallel** at all. That problem was fixed in version 4.4 of the GCC suite, but seems to have slipped back in with some later versions! This may be due to compiler optimizations that place variables in registers.

- Omni: The function **main()** in your executable is actually in the OpenMP library, and your function **main()** is renamed **_ompc_main()**. So, when you enter GDB, first set a breakpoint at your own code:

```
(gdb) b _ompc_main
```

  Then run your program to this breakpoint, and set whatever other breakpoints you want.

  You should find that your other variable and function names are unchanged.

[5] You may find certain subversions of GCC 4.1 can be used too.

- Ompi: Older versions also changed your function names, but the current version (1.2.0) doesn't. Works fine in GDB.

6.10 Performance

As is usually the case with parallel programming, merely parallelizing a program won't necessarily make it faster, even on shared-memory hardware. Operations such as critical sections, barriers and so on serialize an otherwise-parallel program, sapping much of its speed. In addition, there are issues of cache coherency transactions, false sharing etc.

6.10.1 The Effect of Problem Size

To illustrate this, I ran our original Dijkstra example (Section 6.2 on various graph sizes, on a quad core machine. Here are the timings:

nv	nth	time
1000	1	0.005472
1000	2	0.011143
1000	4	0.029574

The more parallelism we had, the *slower* the program ran! The synchronization overhead was just too much to be compensated by the parallel computation.

However, parallelization did bring benefits on larger problems:

nv	nth	time
25000	1	2.861814
25000	2	1.710665
25000	4	1.453052

6.10.2 Some Fine Tuning

How could we make our Dijkstra code faster? One idea would be to eliminate the critical section. Recall that in each iteration, the threads compute their local minimum distance values **md** and **mv**, and then update the global values **md** and **mv**. Since the update must be atomic, this causes some serialization of the program. Instead, we could have the threads store their values **mymd** and **mymv** in a global array **mymins**, with each thread using a separate pair of locations within that array, and then at the end of the iteration we could have just one task scan through **mymins** and update **md** and **mv**.

Here is the resulting code:

```
1  // Dijkstra.c
2
3  // OpenMP example program:  Dijkstra shortest-path finder in a
4  // bidirectional graph; finds the shortest path from vertex 0 to all
5  // others
6
7  // **** in this version, instead of having a critical section in which
8  // each thread updates md and mv, the threads record their mymd and mymv
9  // values in a global array mymins, which one thread then later uses to
10 // update md and mv
11
12 // usage:  dijkstra nv print
13
14 // where nv is the size of the graph, and print is 1 if graph and min
15 // distances are to be printed out, 0 otherwise
16
17 #include <omp.h>
18
19 // global variables, shared by all threads by default
20
21 int nv,  // number of vertices
22     *notdone, // vertices not checked yet
23     nth,  // number of threads
24     chunk,  // number of vertices handled by each thread
25     md,  // current min over all threads
26     mv,  // vertex which achieves that min
27     largeint = -1;  // max possible unsigned int
28
29 int *mymins;  // (mymd,mymv) for each thread; see dowork()
30
31 unsigned *ohd,  // 1-hop distances between vertices; "ohd[i][j]" is
32          // ohd[i*nv+j]
33          *mind;  // min distances found so far
34
35 void init(int ac, char **av)
36 {  int i,j,tmp;
37    nv = atoi(av[1]);
38    ohd = malloc(nv*nv*sizeof(int));
39    mind = malloc(nv*sizeof(int));
40    notdone = malloc(nv*sizeof(int));
41    // random graph
42    for (i = 0; i < nv; i++)
43       for (j = i; j < nv; j++)  {
44          if (j == i) ohd[i*nv+i] = 0;
45          else  {
46             ohd[nv*i+j] = rand() % 20;
47             ohd[nv*j+i] = ohd[nv*i+j];
48          }
49       }
50    for (i = 1; i < nv; i++)  {
51       notdone[i] = 1;
52       mind[i] = ohd[i];
53    }
54 }
55
56 // finds closest to 0 among notdone, among s through e
57 void findmymin(int s, int e, unsigned *d, int *v)
58 {  int i;
59    *d = largeint;
60    for (i = s; i <= e; i++)
```

```
61          if (notdone[i] && mind[i] < *d)  {
62             *d = ohd[i];
63             *v = i;
64          }
65    }
66
67    // for each i in [s,e], ask whether a shorter path to i exists, through
68    // mv
69    void updatemind(int s, int e)
70    {  int i;
71       for (i = s; i <= e; i++)
72          if (mind[mv] + ohd[mv*nv+i] < mind[i])
73             mind[i] = mind[mv] + ohd[mv*nv+i];
74    }
75
76    void dowork()
77    {
78       #pragma omp parallel
79       {  int startv,endv,  // start, end vertices for my thread
80              step,  // whole procedure goes nv steps
81              me,
82              mymv;  // vertex which attains the min value in my chunk
83              unsigned mymd;  // min value found by this thread
84          int i;
85          me = omp_get_thread_num();
86          #pragma omp single
87          {  nth = omp_get_num_threads();
88             if (nv % nth != 0) {
89                printf("nv must be divisible by nth\n");
90                exit(1);
91             }
92             chunk = nv/nth;
93             mymins = malloc(2*nth*sizeof(int));
94          }
95          startv = me * chunk;
96          endv = startv + chunk - 1;
97          for (step = 0; step < nv; step++)  {
98             // find closest vertex to 0 among notdone; each thread finds
99             // closest in its group, then we find overall closest
100            findmymin(startv,endv,&mymd,&mymv);
101            mymins[2*me] = mymd;
102            mymins[2*me+1] = mymv;
103            #pragma omp barrier
104            // mark new vertex as done
105            #pragma omp single
106            {  md = largeint; mv = 0;
107               for (i = 1; i < nth; i++)
108                  if (mymins[2*i] < md) {
109                     md = mymins[2*i];
110                     mv = mymins[2*i+1];
111                  }
112               notdone[mv] = 0;
113            }
114            // now update my section of mind
115            updatemind(startv,endv);
116            #pragma omp barrier
117         }
118      }
119   }
120
121   int main(int argc, char **argv)
```

```
122 {  int i,j,print;
123    double startime,endtime;
124    init(argc,argv);
125    startime = omp_get_wtime();
126    // parallel
127    dowork();
128    // back to single thread
129    endtime = omp_get_wtime();
130    printf("elapsed time:  %f\n",endtime-startime);
131    print = atoi(argv[2]);
132    if (print)  {
133       printf("graph weights:\n");
134       for (i = 0; i < nv; i++)  {
135          for (j = 0; j < nv; j++)
136             printf("%u  ",ohd[nv*i+j]);
137          printf("\n");
138       }
139       printf("minimum distances:\n");
140       for (i = 1; i < nv; i++)
141          printf("%u\n",mind[i]);
142    }
143 }
```

Let's take a look at the latter part of the code for one iteration;

```
1           findmymin(startv,endv,&mymd,&mymv);
2           mymins[2*me] = mymd;
3           mymins[2*me+1] = mymv;
4           #pragma omp barrier
5           // mark new vertex as done
6           #pragma omp single
7           {  notdone[mv] = 0;
8              for (i = 1; i < nth; i++)
9                 if (mymins[2*i] < md) {
10                   md = mymins[2*i];
11                   mv = mymins[2*i+1];
12                }
13          }
14          // now update my section of mind
15          updatemind(startv,endv);
16          #pragma omp barrier
```

The call to **findmymin()** is as before; this thread finds the closest vertex to 0 among this thread's range of vertices. But instead of comparing the result to **md** and possibly updating it and **mv**, the thread simply stores its **mymd** and **mymv** in the global array **mymins**. After all threads have done this and then waited at the barrier, we have just one thread update **md** and **mv**.

Let's see how well this tack worked:

nv	nth	time
25000	1	2.546335
25000	2	1.449387
25000	4	1.411387

This brought us about a 15% speedup in the two-thread case, though less for four threads.

What else could we do? Here are a few ideas:

- False sharing could be a problem here. To address it, we could make **mymins** much longer, changing the places at which the threads write their data, leaving most of the array as padding.
- We could try the modification of our program in Section 6.3.1, in which we use the OpenMP **for** pragma, as well as the refinements stated there, such as **schedule**.
- We could try combining all of the ideas here.

6.10.3 OpenMP Internals

We may be able to write faster code if we know a bit about how OpenMP works inside.

You can get some idea of this from your compiler. For example, if you use the **-t** option with the Omni compiler, or **-k** with Ompi, you can inspect the result of the preprocessing of the OpenMP pragmas.

Here for instance is the code produced by Omni from the call to **findmymin()** in our Dijkstra program:

```
# 93 "Dijkstra.c"
findmymin(startv,endv,&(mymd),&(mymv));{
_ompc_enter_critical(&__ompc_lock_critical);
# 96 "Dijkstra.c"
if((mymd)<(((unsigned )(md)))){

# 97 "Dijkstra.c"
(md)=(((int )(mymd)));
# 97 "Dijkstra.c"
(mv)=(mymv);
}_ompc_exit_critical(&__ompc_lock_critical);
```

Fortunately Omni saves the line numbers from our original source file, but the pragmas have been replaced by calls to OpenMP library functions.

With Ompi, while preprocessing of your file **x.c**, the compiler produces an intermediate file **x_ompi.c**, and the latter is what is actually compiled. Your function **main** is renamed to **_ompi_originalMain()**. Your other functions and variables are renamed. For example in our Dijkstra code, the function **dowork()** is renamed to **dowork_parallel_0**. And by the way, all indenting is lost! So it's a bit hard to read, but can be very instructive.

The document, *The GNU OpenMP Implementation*, http://pl.postech.ac.kr/~gla/cs700-07f/ref/openMp/libgomp.pdf, includes good outline of how the pragmas are translated.

6.11 Example: Root Finding

The application is described in the comments, but here are a couple of things to look for in particular:

- The variables **curra** and **currb** are shared by all the threads, but due to the nature of the application, no critical sections are needed.
- On the other hand, the barrier is essential. The reader should ponder what calamities would occur without it.

Note the disclaimer in the comments, to the effect that parallelizing this application will be fruitful only if the functoin f() is very time-consuming to evaluate. It might be the output of some complex simulation, for instance, with the argument to f() being some simulation parameter.

```
1  #include<omp.h>
2  #include<math.h>
3
4  // OpenMP example:  root finding for a function f() that is continuous
5  // and strictly monotonic; f() is known to be negative
6  // at a, positive at b; thus f() has a root in (a,b)
7
8  // strategy: in each iteration, the current
9  // interval is split into nth equal parts,
10 // and each thread checks its subinterval
11 // for a sign change of f(); if one is
12 // found, this subinterval becomes the
13 // new current interval; the current guess
14 // for the root is the left endpoint of the
15 // current interval
16
17 // of course, this approach is useful in
18 // parallel only if f() is very expensive
19 // to evaluate
20
21 // for simplicity, assumes that no endpoint
22 // of a subinterval will ever exactly
```

```
23 // coincide with a root
24
25 float root(float(*f)(float),
26    float inita, float initb, int niters) {
27    float curra = inita;
28    float currb = initb;
29    #pragma omp parallel
30    {
31       int nth = omp_get_num_threads();
32       int me = omp_get_thread_num();
33       int iter;
34       for (iter = 0; iter < niters; iter++) {
35          #pragma omp barrier
36          float subintwidth =
37             (currb - curra) / nth;
38          float myleft =
39             curra + me * subintwidth;
40          float myright = myleft + subintwidth;
41          // only one thread will find its interval contains the root,
42          // don't need anything lik #pragma omp critical here
43          if ((*f)(myleft) < 0 &&
44              (*f)(myright) > 0) {
45             curra = myleft;
46             currb = myright;
47          }
48       }
49    }
50    return curra;
51 }
52
53 float testf(float x) {
54    return pow(x-2.1,3);
55 }
56
57 int main(int argc, char **argv)
58 {  printf("%f\n",root(testf,-4.1,4.1,1000));  }
```

6.12 Example: Mutual Outlinks

Consider the example of Section 21.4.3. We have a network graph of some kind, such as Web links. For any two vertices, say any two Web sites, we might be interested in mutual outlinks, i.e. outbound links that are common to two Web sites.

The OpenMP code below finds the mean number of mutual outlinks, among all pairs of sites in a set of Web sites. Note that it uses the method for load balancing presented in Section 21.4.3.

```
1  #include <omp.h>
2  #include <stdio.h>
3
4  // OpenMP example:  finds mean number of mutual outlinks, among all
5  // pairs of Web sites in our set
6
7  int n,  // number of sites (will assume n is even)
8      nth,  // number of threads (will assume n/2 divisible by nth)
9      *m, // link matrix
10     tot = 0; // grand total of matches
11
12 // processes row pairs (i,i+1), (i,i+2), ...
13 int procpairs(int i)
14 {  int j,k,sum=0;
15    for (j = i+1; j < n; j++) {
16       for (k = 0; k < n; k++)
17          sum += m[n*i+k] * m[n*j+k];
18    }
19    return sum;
20 }
21
22 float dowork()
23 {
24    #pragma omp parallel
25    {  int pn1,pn2,i,mysum=0;
26       int me = omp_get_thread_num();
27       nth = omp_get_num_threads();
28       // in checking all (i,j) pairs, partition the work according to i;
29       // to get good load balance, this thread me will handle all i
30       // that equal me mod nth
31       for (i = me; i < n; i += nth) {
32          mysum += procpairs(i);
33       }
34       #pragma omp atomic
35       tot += mysum;
36       #pragma omp barrier
37    }
38    int divisor = n * (n-1) / 2;
39    return ((float) tot)/divisor;
40 }
41
42 int main(int argc, char **argv)
43 {   int n2 = n/2,i,j;
44     n = atoi(argv[1]);  // number of matrix rows/cols
45     int msize = n * n * sizeof(int);
46     m = (int *) malloc(msize);
```

```
47      // as a test, fill matrix with random 1s and 0s
48      for (i = 0; i < n; i++) {
49         m[n*i+i] = 0;
50         for (j = 0; j < n; j++) {
51            if (j != i) m[i*n+j] = rand() % 2;
52         }
53      }
54      if (n < 10) {
55         for (i = 0; i < n; i++) {
56            for (j = 0; j < n; j++) printf("%d  ",m[n*i+j]);
57            printf("\n");
58         }
59      }
60      tot = 0;
61      float meanml = dowork();
62      printf("mean = %f\n",meanml);
63  }
```

6.13 Example: Transforming an Adjacency Matrix

Say we have a graph with adjacency matrix

$$\begin{pmatrix} 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{pmatrix} \tag{6.1}$$

with row and column numbering starting at 0, not 1. We'd like to transform this to a two-column matrix that displays the links, in this case

$$\begin{pmatrix} 0 & 1 \\ 1 & 0 \\ 1 & 3 \\ 2 & 1 \\ 2 & 3 \\ 3 & 0 \\ 3 & 1 \\ 3 & 2 \end{pmatrix} \tag{6.2}$$

For instance, there is a 1 on the far right, second row of the above matrix, meaning that in the graph there is an edge from vertex 1 to vertex 3. This results in the row (1,3) in the transformed matrix seen above.

Suppose further that we require this listing to be in lexicographical order, sorted on source vertex and then on destination vertex. Here is code to do this computation in OpenMP:

```
1  // takes a graph adjacency matrix for a directed graph, and converts i
2  // to a 2-column matrix of pairs (i,j), meaning an edge from vertex i
3  // vertex j; the output matrix must be in lexicographical order
4
5  // not claimed efficient, either in speed or in memory usage
6
7  #include <omp.h>
8
9  // needs -lrt link flag for C++
10 #include <time.h>
11 float timediff(struct timespec t1, struct timespec t2)
12 {  if (t1.tv_nsec > t2.tv_nsec) {
13           t2.tv_sec -= 1;
14           t2.tv_nsec += 1000000000;
15    }
16    return t2.tv_sec-t1.tv_sec + 0.000000001 * (t2.tv_nsec-t1.tv_nsec);
17 }
18
19 // transgraph() does this work
20 // arguments:
21 //    adjm:  the adjacency matrix (NOT assumed symmetric), 1 for edge,
22 //           otherwise; note: matrix is overwritten by the function
23 //    n:  number of rows and columns of adjm
24 //    nout:  output, number of rows in returned matrix
25 // return value: pointer to the converted matrix
26 int *transgraph(int *adjm, int n, int *nout)
27 {
28    int *outm,  // to become the output matrix
29        *num1s,  // i-th element will be the number of 1s in row i of a
30        *cumul1s;  // cumulative sums in num1s
31    #pragma omp parallel
32    {  int i,j,m;
33       int me = omp_get_thread_num(),
34           nth = omp_get_num_threads();
35       int myrows[2];
36       int tot1s;
37       int outrow,num1si;
38       #pragma omp single
39       {
40          num1s = malloc(n*sizeof(int));
41          cumul1s = malloc((n+1)*sizeof(int));
```

```
42        }
43        // determine the rows in adjm to be handled by this thread
44        findmyrange(n,nth,me,myrows);
45        // start the action
46        for (i = myrows[0]; i <= myrows[1]; i++) {
47           tot1s = 0;
48           for (j = 0; j < n; j++)
49              if (adjm[n*i+j] == 1) {
50                 adjm[n*i+(tot1s++)] = j;
51              }
52           num1s[i] = tot1s;
53        }
54        #pragma omp barrier
55        #pragma omp single
56        {
57           cumul1s[0] = 0;
58           // now calculate where the output of each row in adjm
59           // should start in outm
60           for (m = 1; m <= n; m++) {
61              cumul1s[m] = cumul1s[m-1] + num1s[m-1];
62           }
63           *nout = cumul1s[n];
64           outm = malloc(2*(*nout) * sizeof(int));
65        }
66        // now fill in this thread's portion
67        for (i = myrows[0]; i <= myrows[1]; i++) {
68           outrow = cumul1s[i];
69           num1si = num1s[i];
70           for (j = 0; j < num1si; j++) {
71              outm[2*(outrow+j)] = i;
72              outm[2*(outrow+j)+1] = adjm[n*i+j];
73           }
74        }
75        #pragma omp barrier
76     }
77     return outm;
78  }
79
80  int main(int argc, char **argv)
81  {  int i,j;
82     int *adjm;
```

```
83     int n = atoi(argv[1]);
84     int nout;
85     int *outm;
86     adjm = malloc(n*n*sizeof(int));
87     for (i = 0; i < n; i++)
88        for (j = 0; j < n; j++)
89           if (i == j) adjm[n*i+j] = 0;
90           else adjm[n*i+j] = rand() % 2;
91
92     struct timespec bgn,nd;
93     clock_gettime(CLOCK_REALTIME, &bgn);
94
95     outm = transgraph(adjm,n,&nout);
96     printf("number of output rows: %d\n",nout);
97
98     clock_gettime(CLOCK_REALTIME, &nd);
99     printf("%f\n",timediff(bgn,nd));
100
101    if (n <= 10)
102       for (i = 0; i < nout; i++)
103          printf("%d %d\n",outm[2*i],outm[2*i+1]);
104 }
105
106 // finds chunk among 0,...,n-1 to assign to thread number me among nth
107 // threads
108 void findmyrange(int n,int nth,int me,int *myrange)
109 {  int chunksize = n / nth;
110    myrange[0] = me * chunksize;
111    if (me < nth-1) myrange[1] = (me+1) * chunksize - 1;
112    else myrange[1] = n - 1;
113 }
```

6.14 Example: Finding the Maximal Burst in a Time Series

Consider a time series of length n, in the context of our example in Section **??**, but with a modified goal, to find the period of at least k consecutive time points that has the maximal mean value.

Denote our time series by $x_1, x_2, ..., x_n$. Consider checking for bursts that begin

at x_i. We could check for bursts of length $k, k+1, ..., n-i+1$, i.e. about $n-i-k$ different cases. Since i itself can take on $O(n)$ values, the time complexity of this application is, for fixed k and varying n, $O(n^2)$. This growth rate in n suggests that this is a good candidate for parallelization.

For convenience, the code will assume that the time series values of nonnegative.

```
1  // OpenMP example program, Burst.c; burst() finds period of
2  // highest burst of activity in a time series
3
4  #include <omp.h>
5  #include <stdio.h>
6  #include <stdlib.h>
7
8  // arguments for burst()
9
10 //    inputs:
11 //       x:  the time series, assumed nonnegative
12 //       nx:  length of x
13 //       k:  shortest period of interest
14 //    outputs:
15 //       startmax, endmax:  pointers to indices of the maximal-burst p
16 //       maxval:  pointer to maximal burst value
17
18 // finds the mean of the block between y[s] and y[e]
19 double mean(double *y, int s, int e) {
20    int i; double tot = 0;
21    for (i = s; i <= e; i++) tot += y[i];
22    return tot / (e - s + 1);
23 }
24
25 void burst(double *x, int nx, int k,
26    int *startmax, int *endmax, double *maxval)
27    {
28    int nth;  // number of threads
29    #pragma omp parallel
30    {  int perstart, // period start
31           perlen,  // period length
32           perend,  // perlen end
33           pl1;  // perlen - 1
34       // best found by this thread so far
35       int mystartmax, myendmax;  // locations
```

```
36          double mymaxval;  // value
37          // scratch variable
38          double xbar;
39          int me;  // ID for this thread
40          #pragma omp single
41          {
42              nth = omp_get_num_threads();
43          }
44          me = omp_get_thread_num();
45          mymaxval = -1;
46          #pragma omp for
47          for (perstart = 0; perstart <= nx-k; perstart++) {
48             for (perlen = k; perlen <= nx - perstart; perlen++) {
49                perend = perstart+perlen-1;
50                if (perlen == k)
51                   xbar = mean(x,perstart,perend);
52                else  {
53                   // update the old mean
54                   pl1 = perlen - 1;
55                   xbar = (pl1 * xbar + x[perend]) / perlen;
56                }
57                if (xbar > mymaxval) {
58                   mymaxval = xbar;
59                   mystartmax = perstart;
60                   myendmax = perend;
61                }
62             }
63          }
64          #pragma omp critical
65          {
66             if (mymaxval > *maxval) {
67                *maxval = mymaxval;
68                *startmax = mystartmax;
69                *endmax = myendmax;
70             }
71          }
72       }
73    }
74
75    // here's our test code
76
```

```
77 int main(int argc, char **argv)
78 {
79    int startmax, endmax;
80    double maxval;
81    double *x;
82    int k = atoi(argv[1]);
83    int i,nx;
84    nx = atoi(argv[2]);  // length of x
85    x = malloc(nx*sizeof(double));
86    for (i = 0; i < nx; i++) x[i] = rand() / (double) RAND_MAX;
87    double startime,endtime;
88    startime = omp_get_wtime();
89    // parallel
90    burst(x,nx,k,&startmax,&endmax,&maxval);
91    // back to single thread
92    endtime = omp_get_wtime();
93    printf("elapsed time:  %f\n",endtime-startime);
94    printf("%d %d %f\n",startmax,endmax,maxval);
95    if (nx < 25) {
96       for (i = 0; i < nx; i++) printf("%f ",x[i]);
97       printf("\n");
98    }
99 }
```

6.15 Locks with OpenMP

Though one of OpenMP's best virtues is that you can avoid working with those pesky lock variables needed for straight threads programming, there are still some instances in which lock variables may be useful. OpenMP does provide for locks:

- declare your locks to be of type `omp_lock_t`
- call `omp_set_lock()` to lock the lock
- call `omp_unset_lock()` to unlock the lock

6.16 Other Examples of OpenMP Code in This Book

There are additional OpenMP examples in later sections of this book, such as:[6]

- sampling bucket sort, Section **??**
- parallel prefix sum/run-length decoding, Section **??**.
- matrix multiplication, Section 20.3.2.
- Jacobi algorithm for solving systems of linear equations, with a good example of the OpenMP **reduction** clause, Section 20.5.4
- another implementation of Quicksort, Section 10.1.2

[6] If you are reading this presentation on OpenMP separately from the book, the book is at `http://heather.cs.ucdavis.edu/~matloff/158/PLN/ParProcBook.pdf`